

Dyn

ami

C

Pro

gra

mm

ing

**Compl
ete
MNC**

Intervi ew

Master class

1D DP .

2D DP .

String DP

.

Knapsack

Family .

Interval DP

*From first
principles
to*

*product-co
mpany
hard
problems
— in C++*

Chapter 0

The
DP

**Thinki
ng**

Framework

Before writing a single line of code, you must build the mental machinery. Every DP problem you will ever see—from easy to Google-hard—follows the same five-step thought process.

The 5-Step Protocol

Step 1 — Recognise overlapping subproblems: Can you express the answer to the original problem in terms of answers to smaller versions of the same problem? If yes, naive recursion will recompute the same sub-answers exponentially many times.

Step 2 — Define the state (most critical step): The 'state' is the exact set of variables that uniquely identifies a subproblem. If $dp[i]$ represents something, write that definition in plain English before touching the keyboard. Wrong state $\rightarrow$ wrong recurrence $\rightarrow$ wrong solution.

Step 3 — Write the recurrence: Express $dp[state]$ purely as a function of smaller states. This is the mathematical heart of the solution.

Step 4 — Identify base cases: Find the smallest states whose answers you know without any recursion. These are the seeds from which the table grows.

Step 5 — Choose implementation + optimise: Top-down (memo) or bottom-up (tabulation)? Then check if space can be reduced.

Top-Do

wn vs

Bottom-

Up —

Mental Model

Top-Down (Memoization): You start at the answer and ask 'what do I need?' You recurse down to smaller subproblems, but cache every result so each subproblem is computed exactly once. Think of it as 'lazy evaluation' — you only compute what you actually need. Write the recursive solution first, add a memo table, done.

Bottom-Up (Tabulation): You start at the base cases and build upward. No recursion stack. You fill a dp table in a carefully chosen order so that when you compute $dp[i]$, every value it depends on is already in the table. This is the production-grade approach — no stack overflow risk, easier to space-optimize.

Space Optimisation: After writing the correct bottom-up solution, examine your recurrence. If $dp[i]$ only depends on $dp[i-1]$ and $dp[i-2]$, you don't need the full array — just two variables. If $dp[i][j]$ only depends on the previous row $dp[i-1][...]$, keep only two rows. This is always the last step, never the first.

✓ Key Insight

The golden rule: get the correct answer first (top-down), then optimize (bottom-up), then space-optimize. Never skip steps.

Chapter 1

1D

Dyna
mic

Progr ammi ng

1D DP problems maintain a single dp array where $dp[i]$ represents the answer to the subproblem defined up to index i . The recurrence looks backward at a window of previous dp values.

Problem 1 — Climbin

g Stairs

(LC 70)

| Easy



Templat e Builder

Problem Statement

You are climbing a staircase with n steps. Each time you can climb 1 or 2 steps. Count the number of distinct ways to reach the top.

Step-by-Step Thought Process

-
1. Overlapping subproblems? To reach step n , you came from step $n-1$ (took 1 step) OR from step $n-2$ (took 2 steps). So $\text{ways}(n) = \text{ways}(n-1) + \text{ways}(n-2)$. Yes, overlapping.
 2. State definition: $\text{dp}[i]$ = number of distinct ways to reach step i .
 3. Recurrence: $\text{dp}[i] = \text{dp}[i-1] + \text{dp}[i-2]$
 4. Base cases: $\text{dp}[1]=1$ (only one way: single step), $\text{dp}[2]=2$ (two ways: 1+1 or 2).
 5. Direction: left to right — $\text{dp}[i]$ needs $\text{dp}[i-1]$ and $\text{dp}[i-2]$, both computed before i .

Top-Down Solution

```
#include <bits/stdc++.h>
using namespace std;

int dp[46]; // memoisation table, -1 = not computed

int climb(int n) {
    if (n == 1) return 1; // base case
    if (n == 2) return 2; // base case
    if (dp[n] != -1) return dp[n]; // already computed
    dp[n] = climb(n-1) + climb(n-2);
    return dp[n];
}

int climbStairs(int n) {
    memset(dp, -1, sizeof(dp));
    return climb(n);
}
```

Bottom- Up Solution

```
int climbStairs(int n) {  
    if (n == 1) return 1;  
    vector<int> dp(n+1);  
    dp[1] = 1;    // base: one way to reach step 1  
    dp[2] = 2;    // base: two ways to reach step 2  
    for (int i = 3; i <= n; i++) {  
        // From step i, you could have come from i-1 or i-2  
        dp[i] = dp[i-1] + dp[i-2];  
    }  
    return dp[n];  
}
```

Space-O ptimised — $O(1)$ Space

```
int climbStairs(int n)
{
    if (n == 1) return
1;
    int prev2 = 1,
prev1 = 2;    //
dp[i-2], dp[i-1]
    for (int i = 3; i
<= n; i++) {
```

```
        int cur = prev1
+ prev2;
        prev2 = prev1;
// slide window
        prev1 = cur;
    }
    return prev1;
}
```

Approach	Time	Space
Top-down (memo)	$O(n)$	$O(n) + O(n)$ stack
Bottom-up (table)	$O(n)$	$O(n)$
Space optimised	$O(n)$	$O(1)$

Proble

m 2 —

House
Robber
(LC

198) |

Medium

|

Classic 1D

Problem Statement

Given an array `nums[]` of non-negative integers representing the amount of money at each house, return the maximum amount you can rob without robbing two adjacent houses.

Thought Process

At each house i , you make a binary choice: rob it or skip it.

- If you ROB house i : you cannot rob $i-1$. Best total = $\text{nums}[i] + \text{dp}[i-2]$
- If you SKIP house i : best total = $\text{dp}[i-1]$

State: $\text{dp}[i]$ = max money robable from houses $0..i$

Recurrence: $\text{dp}[i] = \max(\text{nums}[i] + \text{dp}[i-2], \text{dp}[i-1])$

Base cases: $\text{dp}[0] = \text{nums}[0]$, $\text{dp}[1] = \max(\text{nums}[0], \text{nums}[1])$

Bottom-

Up +

Space

Optimised

```
int rob(vector<int>& nums) {  
    int n = nums.size();  
    if (n == 1) return nums[0];  
    // dp[i] = max money from houses 0..i  
    // We only need the last two values, so skip the full array  
    int prev2 = nums[0];           // dp[i-2]  
    int prev1 = max(nums[0], nums[1]); // dp[i-1]  
    for (int i = 2; i < n; i++) {  
        // Choice: rob i (nums[i]+prev2) vs skip i (prev1)  
        int cur = max(nums[i] + prev2, prev1);  
        prev2 = prev1;  
        prev1 = cur;  
    }  
    return prev1;  
}
```

✓ Key Insight

Pattern lock: whenever your recurrence is $dp[i] = f(dp[i-1], dp[i-2])$, you can always reduce to $O(1)$ space with two rolling variables.

Problem 3 — Longes

t

Increasing
Subseq

uence

(LC

300)

|

**Medium
-Hard**

Problem Statement

Given array `nums[]`, find the length of the longest strictly increasing subsequence.

Thought

Process

— $O(n^2)$

DP

approach

Key insight: for each position i , we consider it as the LAST element of some LIS. The length of the best LIS ending at i is $1 + (\text{best LIS ending at any } j < i \text{ where } \text{nums}[j] < \text{nums}[i])$.

State: $\text{dp}[i]$ = length of longest increasing subsequence ending at index i .

Recurrence: $\text{dp}[i] = 1 + \max(\text{dp}[j])$ for all $j < i$ where $\text{nums}[j] < \text{nums}[i]$. If no such j , $\text{dp}[i] = 1$.

Base case: every $\text{dp}[i]$ starts at 1 (each element alone is an LIS of length 1).

Answer: $\max(\text{dp}[0..n-1])$ — the LIS can end anywhere.

```
// O(n^2) DP
solution
int
lengthOfLIS(vector<int>& nums) {
    int n =
    nums.size();
    vector<int>
    dp(n, 1); //
    every element
    alone = LIS of 1
    int ans = 1;
    for (int i =
    1; i < n; i++) {
        // look
        back at every
        previous index j
        for (int
        j = 0; j < i;
        j++) {
```

```

        if
        (nums[j] <
        nums[i]) {

        // nums[i] can
        extend the LIS
        ending at j

        dp[i] =
        max(dp[i], dp[j]
        + 1);

        }

        ans =
        max(ans, dp[i]);
    }
    return ans;
}

// O(n log n)
Patience sorting
- for interviews
at top companies
int
lengthOfLIS_fast
(vector<int>&
nums) {
    vector<int>
    tails; //
    tails[i] =
    smallest tail of
    all IS of length
    i+1
    for (int x :
    nums) {
        //
        binary search:
        first position
        where tails[pos]
        >= x
        int pos
        =
        lower_bound(tail
        s.begin(),
        tails.end(), x)
        - tails.begin();
        if (pos
        ==
        (int)tails.size(
        ))
        tails.push_back(
        x);
        else
        tails[pos] = x;
        // replace to
        keep tails as
        small as
        possible
    }
}

```

```
}  
    return  
tails.size();  
}
```

Approach	Time	Space	When to use
DP $O(n^2)$	$O(n^2)$	$O(n)$	If $n \leq 2000$, easy to explain
Patience sort $O(n \log n)$	$O(n \log n)$	$O(n)$	Always prefer in MNC interviews

Problem 4 —

**Min
Cost
Climbin
g Stairs**

(LC
746) |
Medium

Problem Statement

Given `cost[]` where `cost[i]` = cost of stepping on stair `i`, once you pay you can climb 1 or 2 stairs. Find min cost to reach the top (beyond last stair).

```
int minCostClimbingStairs(vector<int>& cost) {
    int n = cost.size();
    // dp[i] = min cost to reach step i
    // You can start from step 0 or step 1 for free
    int prev2 = cost[0]; // dp[0]
    int prev1 = cost[1]; // dp[1]
    for (int i = 2; i < n; i++) {
        int cur = cost[i] + min(prev1, prev2);
        prev2 = prev1;
        prev1 = cur;
    }
    // Top is one step beyond the last stair
    // Can arrive from n-1 or n-2
    return min(prev1, prev2);
}
```


Problem 5 — Maximu

m

Product
Subarra
y (LC

152) |
Medium
-Hard

**Tricky
Insight
— Track
Both**

Max AND Min

Products have a special property: a very negative number multiplied by another negative gives a large positive. So the minimum product ending at i could suddenly become the maximum.

State: $\text{maxDP}[i]$ = max product of subarray ending at i . $\text{minDP}[i]$ = min product of subarray ending at i .

```
int maxProduct(vector<int>& nums) {  
    int n = nums.size();  
    int maxProd = nums[0], minProd = nums[0], ans = nums[0];  
    for (int i = 1; i < n; i++) {  
        // If nums[i] is negative, max and min swap!  
        if (nums[i] < 0) swap(maxProd, minProd);  
        // Either start fresh at nums[i] or extend the previous subarray  
        maxProd = max(nums[i], maxProd * nums[i]);  
        minProd = min(nums[i], minProd * nums[i]);  
        ans = max(ans, maxProd);  
    }  
    return ans;  
}
```

Note

Lesson: Not all 1D DP problems track a single quantity. Tracking complementary quantities (max+min, or count+longest) is a key technique for harder 1D problems.

Chapter 2

— 2D

Dyna
mic

Progr ammi ng

2D DP problems require two parameters to uniquely identify a subproblem — typically two indices (i, j) into a grid, string pair, or range. The dp table becomes a 2D matrix and state transitions look at neighboring cells.

Problem 1 — Unique

**Paths
(LC 62)**

|

Medium

| Grid

DP

Entry

Problem Statement

An $m \times n$ grid. A robot starts at top-left corner $(0,0)$ and must reach bottom-right $(m-1, n-1)$. It can only move RIGHT or DOWN. Count the number of unique paths.

Thought Process

To reach cell (i,j), the robot came from (i-1,j) [moved down] OR from (i,j-1) [moved right]. So the number of ways to reach (i,j) is the sum of ways to reach both predecessors.

State: $dp[i][j]$ = number of unique paths to reach cell (i,j)

Recurrence: $dp[i][j] = dp[i-1][j] + dp[i][j-1]$

Base cases: $dp[0][j]=1$ for all j (only one way along top row: all right). $dp[i][0]=1$ for all i (only one way along left col: all down).

```
int uniquePaths(int m, int n) {
    // dp[i][j] = ways to reach cell (i,j)
    vector<vector<int>> dp(m, vector<int>(n, 1));
    // First row and first column already set to 1 by initialisation
    // Fill the rest of the table
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = dp[i-1][j] + dp[i][j-1];
        }
    }
    return dp[m-1][n-1];
}

// Space optimised: only need the previous row
int uniquePaths_opt(int m, int n) {
    vector<int> prev(n, 1); // represents dp[i-1][...]
    for (int i = 1; i < m; i++) {
        vector<int> cur(n, 1);
        for (int j = 1; j < n; j++) {
            cur[j] = prev[j] + cur[j-1]; // from above + from left
        }
        prev = cur;
    }
}
```

```
}  
return prev[n-1];  
}
```

Problem 2 — Unique

Paths II

(LC 63)

|

Medium

| Grid with Obstacles

Same grid, but some cells are obstacles ($\text{grid}[i][j]=1$). A path cannot pass through obstacles.

```
int uniquePathsWithObstacles(vector<vector<int>>& grid) {  
    int m = grid.size(), n = grid[0].size();  
    if (grid[0][0] == 1) return 0; // start blocked  
    vector<vector<long long>> dp(m, vector<long long>(n, 0));
```



```
dp[0][0] = 1;
// Fill first column
for (int i = 1; i < m; i++)
    dp[i][0] = (grid[i][0] == 1) ? 0 : dp[i-1][0];
// Fill first row
for (int j = 1; j < n; j++)
    dp[0][j] = (grid[0][j] == 1) ? 0 : dp[0][j-1];
for (int i = 1; i < m; i++) {
    for (int j = 1; j < n; j++) {
        if (grid[i][j] == 1) dp[i][j] = 0; // obstacle
        else dp[i][j] = dp[i-1][j] + dp[i][j-1];
    }
}
return dp[m-1][n-1];
}
```

Problem 3 —

Minimum Path Sum (LC 64)

I Medium

Grid of non-negative integers. Find path from top-left to bottom-right (only right/down) with minimum sum.

```
int minPathSum(vector<vector<int>>& grid) {  
    int m = grid.size(), n = grid[0].size();  
    // Modify grid in-place as dp table (saves space)  
    for (int i = 0; i < m; i++) {  
        for (int j = 0; j < n; j++) {  
            if (i==0 && j==0) continue; // start cell unchanged  
            else if (i==0) grid[i][j] += grid[i][j-1]; // top row: only  
from left  
            else if (j==0) grid[i][j] += grid[i-1][j]; // left col: only  
from above  
            else grid[i][j] += min(grid[i-1][j], grid[i][j-1]); // min of  
above/left  
        }  
    }  
    return grid[m-1][n-1];  
}
```

Problem 4 — Triangle

(LC

120) |

Medium

|

**Bottom-
Up Gem**

**Problem
State**

DP + Key Insight

Given a triangle array, find the minimum path sum from top to bottom. Each step you may move to adjacent numbers on the row below.

Key insight: Go BOTTOM-UP. Start at the bottom row. For each cell $dp[i][j]$, its best continuation is $\min(dp[i+1][j], dp[i+1][j+1])$. Roll upward until $dp[0][0]$ holds the answer. No extra space needed — modify the triangle.

```
int minimumTotal(vector<vector<int>>& tri) {
    int n = tri.size();
    // Start from second-to-last row and go upward
    for (int i = n-2; i >= 0; i--) {
        for (int j = 0; j <= i; j++) {
            // Best move from (i,j) is to whichever of the two adjacent
            // cells below is smaller
            tri[i][j] += min(tri[i+1][j], tri[i+1][j+1]);
        }
    }
    return tri[0][0]; // propagated answer
}
```

Problem 5 — Maxima

I
Square
(LC
221) |

**Medium
-Hard |
Non-Ob**

**vious
State**

The

Non-Ob

vious State Definitio n

$dp[i][j]$ = side length of the largest square whose BOTTOM-RIGHT corner is at (i,j) .

Recurrence: if $matrix[i][j] == '1'$, then $dp[i][j] = 1 + \min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1])$. The minimum of the three neighbors forces all three sub-squares to exist simultaneously.

If $matrix[i][j] == '0'$, $dp[i][j] = 0$ (no square can end here).

```
int maximalSquare(vector<vector<char>>& matrix) {  
    int m = matrix.size(), n = matrix[0].size();  
    vector<vector<int>> dp(m, vector<int>(n, 0));
```

```

int ans = 0;
for (int i = 0; i < m; i++) {
    for (int j = 0; j < n; j++) {
        if (matrix[i][j] == '0') { dp[i][j] = 0; continue; }
        if (i == 0 || j == 0) {
            dp[i][j] = 1; // border cells can only be 1x1 squares
        } else {
            // All three neighbours must support a (k-1) sized square
            dp[i][j] = 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
        }
        ans = max(ans, dp[i][j] * dp[i][j]); // area = side^2
    }
}
return ans;
}

```

✓ Key Insight

Why min of three neighbors? The square ending at (i,j) with side k requires: a (k-1) square ending at (i-1,j), a (k-1) square ending at (i,j-1), and a (k-1) square ending at (i-1,j-1). The bottleneck is the smallest of these three.

Chapter 3

**String
Dyna**

mic

Progr

ammi ng

String DP problems involve one or two strings and typically ask about subsequences, substrings, edit distances, or transformations. The 2D dp table has dimensions $\text{len}(s1)+1 \times \text{len}(s2)+1$. Index i represents the prefix $s1[0..i-1]$.

Problem 1 — Longes

t

Common

n

Subseq

uence

(LC

1143) |

Medium

I Core Pattern

Problem Statement

Given two strings `text1` and `text2`, return the length of their longest common subsequence. A subsequence is derived by deleting some characters (possibly zero) without changing the relative order.

Complet e Thought Process

State: $dp[i][j]$ = length of LCS of $text1[0..i-1]$ and $text2[0..j-1]$.

Recurrence:

- If $text1[i-1] == text2[j-1]$: characters match! LCS extends: $dp[i][j] = dp[i-1][j-1] + 1$
- If they don't match: LCS is the best we can get by skipping either character: $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$

Base cases: $dp[0][j]=0$ for all j (empty string has LCS 0), $dp[i][0]=0$ for all i .

```

int longestCommonSubsequence(string t1, string t2) {
    int m = t1.size(), n = t2.size();
    // dp[i][j] = LCS of t1[0..i-1] and t2[0..j-1]
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (t1[i-1] == t2[j-1]) {
                dp[i][j] = dp[i-1][j-1] + 1; // characters match: extend
            } else {
                // Skip char from t1 OR skip char from t2, take the better
                dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
            }
        }
    }
    return dp[m][n];
}

// Space optimised: dp[i][j] only needs dp[i-1][...] (previous row)
int lcs_opt(string t1, string t2) {
    int m = t1.size(), n = t2.size();
    vector<int> prev(n+1, 0), cur(n+1, 0);
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (t1[i-1] == t2[j-1]) cur[j] = prev[j-1] + 1;
            else cur[j] = max(prev[j], cur[j-1]);
        }
        prev = cur;
        fill(cur.begin(), cur.end(), 0);
    }
    return prev[n];
}

```


Problem 2 —

Edit

Distance (LC
72) |
Hard |

**Most
Importa
nt**

String DP

Problem Statement

Given two strings word1 and word2, return the minimum number of operations (insert, delete, replace) to convert word1 to word2.

Deep Analysis

State: $dp[i][j]$ = min operations to convert $word1[0..i-1]$ to $word2[0..j-1]$.

Recurrence — think about what we do at position i, j :

- $word1[i-1] == word2[j-1]$: characters match, no operation needed. $dp[i][j] = dp[i-1][j-1]$
- Characters differ — three choices:
 - Replace $word1[i-1]$ with $word2[j-1]$: $dp[i][j] = dp[i-1][j-1] + 1$
 - Delete $word1[i-1]$ (move i forward): $dp[i][j] = dp[i-1][j] + 1$
 - Insert $word2[j-1]$ into $word1$ (move j forward): $dp[i][j] = dp[i][j-1] + 1$
- Take the minimum of all three options.

Base cases: $dp[i][0]=i$ (delete all i chars), $dp[0][j]=j$ (insert j chars).

```
int minDistance(string w1, string w2) {
    int m = w1.size(), n = w2.size();
    // dp[i][j] = min operations to convert w1[0..i-1] to w2[0..j-1]
    vector<vector<int>> dp(m+1, vector<int>(n+1, 0));
    // Base cases
    for (int i = 0; i <= m; i++) dp[i][0] = i; // delete all
    for (int j = 0; j <= n; j++) dp[0][j] = j; // insert all
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (w1[i-1] == w2[j-1]) {
                dp[i][j] = dp[i-1][j-1]; // match: free
            } else {
                dp[i][j] = 1 + min({
                    dp[i-1][j-1], // replace
                    dp[i-1][j],    // delete from w1
                    dp[i][j-1]     // insert into w1
                });
            }
        }
    }
}
```

```
    }  
    }  
    }  
    return dp[m][n];  
}
```

Problem 3 —

Longes

t

Palindr

omic

Subseq
uence
(LC

516) | Medium

**Key
Insight**

Reduce to LCS

The longest palindromic subsequence of string s equals the LCS of s and its reverse. Because a palindrome reads the same forwards and backward, matching s with its reverse finds the longest symmetric structure.

```
int longestPalindromeSubseq(string s) {
    string rev = s;
    reverse(rev.begin(), rev.end());
    // LCS of s and its reverse = longest palindromic subsequence
    int n = s.size();
    vector<vector<int>> dp(n+1, vector<int>(n+1, 0));
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= n; j++) {
            if (s[i-1] == rev[j-1]) dp[i][j] = dp[i-1][j-1] + 1;
            else dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
        }
    return dp[n][n];
}
```

Problem 4 — Distinct

Subsequences (LC

115) |

Hard

State

and

Recurrence

Count how many times t appears as a subsequence in s .

State: $dp[i][j]$ = number of ways to form $t[0..j-1]$ using $s[0..i-1]$.

Recurrence: At $s[i-1]$ and $t[j-1]$, you either USE $s[i-1]$ to match $t[j-1]$ (if they're equal) or you SKIP $s[i-1]$. $dp[i][j] = dp[i-1][j]$ (always skip) + $(s[i-1] == t[j-1] ? dp[i-1][j-1] : 0)$ (use if match).

```
int numDistinct(string s, string t) {
    int m = s.size(), n = t.size();
    // dp[i][j] = ways to form t[0..j-1] using s[0..i-1]
    vector<vector<long long>> dp(m+1, vector<long long>(n+1, 0));
    for (int i = 0; i <= m; i++) dp[i][0] = 1; // empty t, always 1 way
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            dp[i][j] = dp[i-1][j]; // skip s[i-1]
            if (s[i-1] == t[j-1])
                dp[i][j] += dp[i-1][j-1]; // also use s[i-1]
        }
    }
    return dp[m][n];
}
```

Problem 5 — Palindr

omnic
Substri
ngs &
Min Cut

(LC
647,
132) |

**Medium
-Hard**

**Building
the**

isPalin table first

$\text{isPalin}[i][j] = \text{true}$ if $s[i..j]$ is a palindrome. This table is used by many interval DP problems. Build it for all intervals of increasing length.

```
// Precompute isPalin[i][j] for string s
// Then use it for Min Cut Palindrome Partitioning
int minCut(string s) {
    int n = s.size();
    vector<vector<bool>> isPalin(n, vector<bool>(n, false));
    // Every single char is a palindrome
    for (int i = 0; i < n; i++) isPalin[i][i] = true;
    // Check pairs
    for (int i = 0; i < n-1; i++)
        isPalin[i][i+1] = (s[i] == s[i+1]);
    // Check longer substrings
    for (int len = 3; len <= n; len++)
        for (int i = 0; i <= n-len; i++) {
            int j = i + len - 1;
            isPalin[i][j] = (s[i]==s[j]) && isPalin[i+1][j-1];
        }
    // minCut[i] = min cuts needed for s[0..i]
    vector<int> cut(n, 0);
    for (int i = 0; i < n; i++) {
```

```
    if (isPalin[0][i]) { cut[i] = 0; continue; }
    cut[i] = i; // worst case: cut before every char
    for (int j = 1; j <= i; j++) {
        if (isPalin[j][i])
            cut[i] = min(cut[i], cut[j-1] + 1);
    }
}
return cut[n-1];
}
```

Chapter 4

The Knaps

ack Famil y

The Knapsack family is the most critical DP pattern for product company interviews. It models resource allocation problems: given a budget (weight/capacity), maximise value. Understanding the three variants and their differences is essential.

The Three Variants

at a Glance

Variant	Each item	Recurrence direction	Classic problems
0/1 Knapsack	Used 0 or 1 times	j from W down to w[i]	Partition Equal Subset
Unbounded Knapsack	Used unlimited times	j from w[i] up to W	Coin Change, Rod Cutting
Bounded/Subset Sum	Exists or doesn't	Depends on constraint	Subset Sum, Count partitions

Proble

m 1 —

0/1

**Knapsa
ck |
Core**

**Templat
e**

Problem Statement

Given n items each with weight $w[i]$ and value $v[i]$, and a knapsack of capacity W , find the maximum value achievable without exceeding the capacity. Each item can be used AT MOST once.

State Definition n and

Recurrence

State: $dp[i][j]$ = max value using items 0..i-1 with capacity j.

At each item i, two choices:

- SKIP item i: $dp[i][j] = dp[i-1][j]$
- TAKE item i (only if $j \geq w[i]$): $dp[i][j] = v[i] + dp[i-1][j-w[i]]$

$dp[i][j]$ = max of both choices.

Base cases: $dp[0][j]=0$ for all j, $dp[i][0]=0$ for all i.

```
// Full 2D 0/1 Knapsack
int knapsack01(vector<int>& w, vector<int>& v, int W) {
    int n = w.size();
    vector<vector<int>> dp(n+1, vector<int>(W+1, 0));
    for (int i = 1; i <= n; i++) {
        for (int j = 0; j <= W; j++) {
            dp[i][j] = dp[i-1][j]; // skip item i
            if (j >= w[i-1]) // can we fit item i?
                dp[i][j] = max(dp[i][j], v[i-1] + dp[i-1][j-w[i-1]]);
        }
    }
    return dp[n][W];
}

// ★ Space optimised 0/1 Knapsack - traverse j RIGHT-TO-LEFT
// Why right to left? dp[j-w[i]] must come from the PREVIOUS row (item i-1)
// If we go left to right, we'd pick item i more than once!
int knapsack01_opt(vector<int>& w, vector<int>& v, int W) {
    int n = w.size();
    vector<int> dp(W+1, 0);
    for (int i = 0; i < n; i++) {
        for (int j = W; j >= w[i]; j--) { // RIGHT TO LEFT - critical!
            dp[j] = max(dp[j], v[i] + dp[j-w[i]]);
        }
    }
    return dp[W];
}
```



```
        dp[j] = max(dp[j], v[i] + dp[j - w[i]]);  
    }  
    }  
    return dp[W];  
}
```

⚠ Important

The right-to-left sweep in 1D 0/1 knapsack is the most commonly asked interview question about this pattern. It prevents an item from being counted multiple times in a single iteration.

Problem 2 —

Subset
Sum |
Boolea

n

Variant

Problem Statement

Given an array `nums[]`, determine if any subset sums to target `S`.

State: `dp[i][j]` = true if any subset of `nums[0..i-1]` sums to `j`.

Recurrence: `dp[i][j] = dp[i-1][j]` (don't take `nums[i-1]`) OR `dp[i-1][j-nums[i-1]]` if `j >= nums[i-1]` (take it).

```
bool subsetSum(vector<int>& nums, int S) {
    int n = nums.size();
    // dp[j] = can we form sum j using processed elements
    vector<bool> dp(S+1, false);
    dp[0] = true; // sum 0 is always achievable (empty subset)
    for (int i = 0; i < n; i++) {
        for (int j = S; j >= nums[i]; j--) { // right to left (0/1
            knapsack) dp[j] = dp[j] || dp[j - nums[i]];
        }
    }
    return dp[S];
}
```

Problem 3 — Partitio

n Equal
Subset
Sum
(LC

416)

|

Medium

|

Commo

n

Interview

w

Reduction to Subset Sum

Can we partition `nums[]` into two subsets of equal sum? If total sum is odd, impossible. Otherwise, find if any subset sums to `total/2`.

```
bool canPartition(vector<int>& nums) {  
    int total = 0;  
    for (int x : nums) total += x;  
    if (total % 2 != 0) return false; // odd total - impossible  
    int target = total / 2;  
    // Subset sum problem: can we pick numbers summing to target?
```

```
vector<bool> dp(target+1, false);  
dp[0] = true;  
for (int x : nums) {  
    for (int j = target; j >= x; j--) // right to left  
        dp[j] = dp[j] || dp[j-x];  
}  
return dp[target];  
}
```

Problem 4 —

Unboun
ded
Knapsa
ck |

**Items
can be
reused**

Key Difference from 0/1

Each item can be selected ANY number of times. This changes only one thing in the implementation: we traverse j from LEFT TO RIGHT (small to large), allowing the same item to be picked again in the same iteration.

```
// Unbounded Knapsack - each item usable unlimited times
int unboundedKnapsack(vector<int>& w, vector<int>& v, int W) {
    int n = w.size();
    vector<int> dp(W+1, 0);
    for (int i = 0; i < n; i++) {
```

```
for (int j = w[i]; j <= W; j++) { // LEFT TO RIGHT – critical!  
    // When computing dp[j], dp[j-w[i]] already reflects  
    // taking item i before – so item i can be taken again  
    dp[j] = max(dp[j], v[i] + dp[j - w[i]]);  
}  
}  
return dp[W];  
}
```

Problem 5 —

Coin
Change
(LC
322) |

**Medium
| Must
Know**

Minimum m coins variant

unbounded

ded

Given coin denominations and a target amount, find the minimum number of coins to make that amount. Each coin can be used unlimited times.

```
int coinChange(vector<int>& coins, int amount) {  
    // dp[j] = min coins to make amount j  
    vector<int> dp(amount+1, amount+1); // initialise to 'infinity'  
    dp[0] = 0; // 0 coins needed to make amount 0  
    for (int coin : coins) {  
        for (int j = coin; j <= amount; j++) { // left to right (unbounded)  
            // either skip this coin, or use it (1 + dp[j-coin])  
            dp[j] = min(dp[j], 1 + dp[j - coin]);  
        }  
    }  
    return dp[amount] > amount ? -1 : dp[amount];  
}
```

Problem 6 — Coin

Change

II (LC

518) |

Medium

| Count
combin
ations

**Count
the
ways —
order of**

loops matters!

Count the number of distinct combinations. The key difference: loop over COINS in outer loop and AMOUNTS in inner loop. This ensures each combination is counted once regardless of order.

```
int change(int amount, vector<int>& coins) {
    // dp[j] = number of ways to make amount j
    vector<long long> dp(amount+1, 0);
    dp[0] = 1; // one way to make 0: use no coins
    // OUTER loop over coins ensures combinations (not permutations)
    // If we looped amounts outside, [1,2] and [2,1] would both count
    for (int coin : coins) {
        for (int j = coin; j <= amount; j++) { // left to right (unbounded)
            dp[j] += dp[j - coin];
        }
    }
    return dp[amount];
}
```

⚠ Important

Critical rule: For combination count (order doesn't matter), coin/item outer loop + amount inner loop. For permutation count (order matters), amount outer loop + coin inner loop.

Problem 7 — Target

Sum

(LC

494) |

Medium

**| Count
subsets**

**Reducti
on to**

**Count
Subsets
with**

Given Sum

Assign + or - to each number. Count ways to reach target. If P = positive subset sum, N = negative subset sum, then $P + N = \text{total}$ and $P - N = \text{target}$. Therefore $P = (\text{total} + \text{target}) / 2$. Count subsets summing to P.

```
int findTargetSumWays(vector<int>& nums, int target) {
    int total = 0;
    for (int x : nums) total += x;
    // Must be reachable: |target| <= total, and (total+target) must be even
    if (abs(target) > total || (total + target) % 2 != 0) return 0;
    int S = (total + target) / 2; // target subset sum
    // Count number of subsets summing to S
    vector<int> dp(S+1, 0);
    dp[0] = 1;
    for (int x : nums) {
        for (int j = S; j >= x; j--) // right to left (0/1 - each used
once)
            dp[j] += dp[j - x];
    }
    return dp[S];
}
```

Chapter 5



Interval

**Dyna
mic
Progr**

ammi ng

Interval DP solves problems on contiguous segments (intervals) of an array or string. The state is $dp[i][j]$ = answer for the subarray/substring from index i to j . We build from small intervals to large, always combining already-solved smaller intervals.

The Universal Interval

DP Templat e

```
// Universal interval DP structure
// 1. Loop over all interval lengths (len = 1 to n)
// 2. For each length, try all starting points i
// 3. j = i + len - 1 is the ending point
// 4. Try all split points k in [i, j-1]
// 5. Combine dp[i][k] and dp[k+1][j]

for (int len = 2; len <= n; len++) {
    for (int i = 0; i <= n - len; i++) {
        int j = i + len - 1;
        dp[i][j] = INF; // or 0 depending on problem
        for (int k = i; k < j; k++) {
            dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] + cost(i,k,j));
        }
    }
}
```

Problem 1 — Matrix

Chain Multipli cation | Classic

Interval DP

Problem Statement

Given n matrices with dimensions $p[0] \times p[1]$, $p[1] \times p[2]$, ..., $p[n-1] \times p[n]$, find the minimum number of scalar multiplications to compute the product of all matrices.

Deep Analysis

Multiplying an $a \times b$ matrix with a $b \times c$ matrix costs $a \cdot b \cdot c$ multiplications. The order of parenthesisation does not change the result but drastically changes the cost. We try every possible last multiplication (split point k).

State: $dp[i][j]$ = minimum multiplications to compute matrix product from i to j .

Recurrence: $dp[i][j] = \min \text{ over all } k \text{ in } [i, j-1] \text{ of } dp[i][k] + dp[k+1][j] + p[i] \cdot p[k+1] \cdot p[j+1]$.

Base case: $dp[i][i] = 0$ (single matrix needs 0 multiplications).

```
int
matrixChainOrder(vector
<int>& p) {
    int n = p.size() -
1; // number of
matrices
    // dp[i][j] = min
cost to multiply
matrices i through j
    vector<vector<int>>
dp(n, vector<int>(n,
0));
    // len = number of
matrices in the chain
    for (int len = 2;
len <= n; len++) {
        for (int i = 0;
i <= n - len; i++) {
            int j = i +
len - 1;
            dp[i][j] =
INT_MAX;
```

```

        // Try
        every split point k:
        (i..k) * (k+1..j)
        for (int k
= i; k < j; k++) {
            int
cost = dp[i][k] +
dp[k+1][j]

+ p[i] * p[k+1] *
p[j+1];

dp[i][j] =
min(dp[i][j], cost);
        }
    }
    return dp[0][n-1];
}

```

	Time	Space
Matrix Chain	$O(n^3)$	$O(n^2)$

Proble

m 2 —

Burst

Balloon

s (LC

312) |
Hard |
Revers
e

**Thinkin
g**

**The
Counter-**

**Intuitive
Trick —
Think
Last**

Burst, Not First

If we think about which balloon to burst first, the subproblems are not independent (bursting a balloon changes the neighbors of remaining balloons).

Reverse the thinking: for the interval $[i, j]$, which balloon k is the LAST one to be burst? When k is burst last, all other balloons in $[i, j]$ are already gone, so the cost of bursting k is $\text{nums}[i-1] * \text{nums}[k] * \text{nums}[j+1]$. The left and right parts $[i, k-1]$ and $[k+1, j]$ are independent because k is still present when they are solved.

State: $\text{dp}[i][j]$ = max coins from bursting all balloons in the open interval (i, j) (excluding i and j which are the boundaries).

Recurrence: $\text{dp}[i][j] = \max \text{ over all } k \text{ in } (i, j) \text{ of } \text{dp}[i][k] + \text{dp}[k][j] + \text{nums}[i] * \text{nums}[k] * \text{nums}[j]$.

```
int maxCoins(vector<int>& nums) {
    // Add virtual balloons of value 1 at both ends
    nums.insert(nums.begin(), 1);
    nums.push_back(1);
    int n = nums.size();
    // dp[i][j] = max coins from bursting all in open interval (i,j)
    vector<vector<int>> dp(n, vector<int>(n, 0));
    // len is the gap size (j - i)
    for (int len = 2; len < n; len++) {
        for (int i = 0; i < n - len; i++) {
            int j = i + len;
            // Try each k as the LAST balloon to burst in (i,j)
            for (int k = i+1; k < j; k++) {
                int coins = nums[i] * nums[k] * nums[j];
                dp[i][j] = max(dp[i][j], dp[i][k] + coins + dp[k][j]);
            }
        }
    }
    return dp[0][n-1];
}
```

```
}
```

✓ Key Insight

Burst Balloons is a model for many 'reverse-thinking' DP problems. When direct simulation creates dependencies, ask: what if I think about the LAST action rather than the first?

Problem 3 —

Strange Printer (LC

664) |

Hard

**Problem
State**

nt +

Insight

A printer can print a sequence of same characters in one turn and can overprint previous characters. Find minimum turns to print string s .

State: $dp[i][j]$ = min turns to print $s[i..j]$.

Base: $dp[i][i] = 1$.

Recurrence: $dp[i][j] = dp[i][j-1] + 1$ (print $s[j]$ alone as an extra turn). But if $s[k] == s[j]$ for any k in $[i, j-1]$, the last character of $s[j]$ can be 'merged' into the print of $s[k]$: $dp[i][j] = \min(dp[i][j], dp[i][k] + dp[k+1][j] - 1)$.

```
int strangePrinter(string s) {
    int n = s.size();
    vector<vector<int>> dp(n, vector<int>(n, 0));
    // Build for increasing lengths
    for (int i = n-1; i >= 0; i--) {
        dp[i][i] = 1;
        for (int j = i+1; j < n; j++) {
            // Worst case: print s[j] in a new turn
            dp[i][j] = dp[i][j-1] + 1;
            // If any s[k] == s[j], we can extend that print to cover j
            for (int k = i; k < j; k++) {
                if (s[k] == s[j]) {
                    // Merge s[j] into the print starting at k
                    dp[i][j] = min(dp[i][j], dp[i][k] + dp[k+1][j] - 1);
                }
            }
        }
    }
    return dp[0][n-1];
}
```

Problem 4 — Minimu

m Cost
Tree /
Optimal
BST |

Interval DP on Trees

Problem Statement

Given n keys with search frequencies $\text{freq}[]$, build a BST minimising the total search cost. Search cost = sum of $(\text{depth}+1) \times \text{freq}[i]$.

State: $\text{dp}[i][j]$ = minimum cost of BST built from keys i to j . $\text{sum}[i][j]$ = $\text{freq}[i] + \dots + \text{freq}[j]$ (precomputed prefix sum).

Recurrence: $\text{dp}[i][j] = \min \text{ over } k \text{ in } [i, j] \text{ of } \text{dp}[i][k-1] + \text{dp}[k+1][j] + \text{sum}[i][j]$. The $\text{sum}[i][j]$ term accounts for the fact that all keys in the subtree go one level deeper when we add the root.

```
int optimalBST(vector<int>& freq) {
    int n = freq.size();
    // prefix sums for range sum queries
    vector<int> pre(n+1, 0);
    for (int i = 0; i < n; i++) pre[i+1] = pre[i] + freq[i];
    auto rangeSum = [&](int i, int j) { return pre[j+1] - pre[i]; };
    vector<vector<int>> dp(n, vector<int>(n, 0));
    // base: single keys
    for (int i = 0; i < n; i++) dp[i][i] = freq[i];
    for (int len = 2; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
```

```
int rs = rangeSum(i, j);
for (int k = i; k <= j; k++) {
    int left = (k > i) ? dp[i][k-1] : 0;
    int right = (k < j) ? dp[k+1][j] : 0;
    dp[i][j] = min(dp[i][j], left + right + rs);
}
}
return dp[0][n-1];
}
```

Problem 5 —

**Minimum
Cost
to
Merge**

Stones

(LC

1000) |

Hard

Problem Statement

There are n piles of stones. Each turn, merge exactly k consecutive piles into one (cost = sum of merged piles). Return minimum cost to merge all piles into 1. If impossible, return -1.

Feasibility: Merging is only possible if $(n-1) \% (k-1) == 0$.

State: $dp[i][j]$ = min cost to reduce $s[i..j]$ to as few piles as possible.

```
int mergeStones(vector<int>& stones, int k) {
    int n = stones.size();
    if ((n-1) % (k-1) != 0) return -1; // impossible
    // prefix sums for O(1) range sum
    vector<int> pre(n+1, 0);
    for (int i = 0; i < n; i++) pre[i+1] = pre[i] + stones[i];
    auto rsum = [&](int i, int j){ return pre[j+1] - pre[i]; };
    vector<vector<int>> dp(n, vector<int>(n, 0));
    for (int len = k; len <= n; len++) {
        for (int i = 0; i <= n - len; i++) {
            int j = i + len - 1;
            dp[i][j] = INT_MAX;
            // Split into a left part and right part
            // Left part has (m-1) % (k-1) == 0 piles after merging
            for (int mid = i; mid < j; mid += k-1) {
```

```
        if (dp[i][mid] == INT_MAX || dp[mid+1][j] == INT_MAX)
continue;
        dp[i][j] = min(dp[i][j], dp[i][mid] + dp[mid+1][j]);
    }
    // If the whole range can be merged to 1 pile, add the cost
    if ((len - 1) % (k-1) == 0)
        dp[i][j] += rsum(i, j);
}
}
return dp[0][n-1];
}
```

Problem 6 —

Interlea ving String (LC 97)

|

Medium

-Hard

|

String Interval

Problem Statement

Given s_1 , s_2 , s_3 , determine if s_3 can be formed by interleaving s_1 and s_2 .

State: $dp[i][j]$ = true if $s_3[0..i+j-1]$ can be formed by interleaving $s_1[0..i-1]$ and $s_2[0..j-1]$.

```
bool isInterleave(string s1, string s2, string s3) {
    int m = s1.size(), n = s2.size();
    if (m + n != (int)s3.size()) return false;
    vector<vector<bool>> dp(m+1, vector<bool>(n+1, false));
    dp[0][0] = true;
    for (int i = 1; i <= m; i++)
        dp[i][0] = dp[i-1][0] && s1[i-1] == s3[i-1];
    for (int j = 1; j <= n; j++)
        dp[0][j] = dp[0][j-1] && s2[j-1] == s3[j-1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            // s3[i+j-1] matches last of s1 used, and prev dp valid
            bool fromS1 = dp[i-1][j] && s1[i-1] == s3[i+j-1];
            bool fromS2 = dp[i][j-1] && s2[j-1] == s3[i+j-1];
            dp[i][j] = fromS1 || fromS2;
        }
    }
    return dp[m][n];
}
```


Chapter 6



Intervi ew

**Strate
gy &
Quick**

Reference

How to
Identify

the DP
Type in
an

Interview

W

If the problem has...	Think...	Pattern
Single index, look-back recurrence	1D DP	House Robber, LIS, Kadane
Grid / two indices, move right+down	2D Grid DP	Unique Paths, Min Path Sum
Two strings, match/skip choices	String DP	LCS, Edit Distance
Budget/capacity constraint	Knapsack	0/1 vs Unbounded
Subarray/substring, combine intervals	Interval DP	Burst Balloons, MCM
Count ways / combinations	Count DP	Coin Change II, Target Sum

The Direction Rule — Most

Common Mistake

Situation	Inner loop direction	Reason
0/1 Knapsack (each item once)	$j = W$ down to $w[i]$	Prevents reuse in same pass
Unbounded Knapsack (reuse ok)	$j = w[i]$ up to W	Allows reuse in same pass
LCS / Edit Distance / 2D DP	Outer i left→right, inner j left→right	Build on smaller prefixes
Interval DP	Outer = length, inner = start	Small intervals before large

**State
Space
Complexity**

Referen ce

Problem type	State dimensions	Typical complexity
1D DP	<code>dp[i]</code>	$O(n)$ time, $O(n) \rightarrow O(1)$ space
2D Grid DP	<code>dp[i][j]</code>	$O(m \times n)$ time, $O(n)$ space w/ opt
Knapsack	<code>dp[n][W]</code>	$O(n \times W)$ time, $O(W)$ space w/ opt
String DP	<code>dp[m][n]</code>	$O(m \times n)$ time, $O(n)$ space w/ opt
Interval DP	<code>dp[i][j]</code> , len 1..n	$O(n^3)$ time, $O(n^2)$ space

✓ Key Insight

Final Interview Tip: Always verbally define your state in English before writing code. Say: '`dp[i]` represents...' or '`dp[i][j]` represents...'. This demonstrates clarity of thought and instantly shows you understand DP thinking rather than just pattern matching.

Common Pitfalls Checklist

-
- ☐ Did I initialise the dp table correctly? (INF for min, 0 for count, false for boolean)
 - ☐ Are my base cases covering all boundary conditions?
 - ☐ Is my loop order correct? (interval DP: length outside, start inside)
 - ☐ For 0/1 knapsack: am I traversing right to left?
 - ☐ For unbounded knapsack: am I traversing left to right?
 - ☐ For interval DP: is $dp[i][i]$ (or $dp[i][i+1]$) the correct base?

- Am I returning $dp[n]$ or $dp[n-1]$? (depends on 0-indexed vs 1-indexed state)
- After solving, can I reduce from 2D table to 1D rolling array?